NBT Price Comparison Platform

None

Developer Team

None

Table of contents

1. NBT Price Comparison Documentation 3
   - 1.1 Getting Started 3
   - 1.2 Project Overview 3
2. Setup Instructions 4
   - 2.1 Prerequisites 4
   - 2.2 Installation Steps 4
   - 2.3 Running the Application 4
3. Architecture Overview 6
   - 3.1 Tech Stack 6
   - 3.2 Directory Structure 6
4. Features Overview 8
   - 4.1 1. Localization & RTL 8
   - 4.2 2. Financial Products Catalog 8
   - 4.3 3. Dynamic Calculators 8
   - 4.4 4. Exchange Rates Manager 8
   - 4.5 5. ATMs & Branches Locator 8
   - 4.6 6. Ratings & Application Submission (Reviews) 8
   - 4.7 7. Role-Based Admin Panel 8
5. API & Routing 9
   - 5.1 Routing Architecture 9
   - 5.2 Client-Side State & SSR 9
   - 5.3 Third-Party Scripts & API Exts 9

1. NBT Price Comparison Documentation

Welcome to the **Developer Documentation** for the Centralized website for price/offer comparison.

This platform empowers users to compare various financial products such as credits, insurance, exchange rates, and view bank ratings, along with tools like loan and deposit calculators.

1.1 Getting Started

If you are a new developer on the project, please start with the Setup Guide to initialize your local environment.

To understand the core underlying technologies, visit the Architecture Overview.

1.2 Project Overview

- Built with modern frontend technologies including **Vue 3**, **Inertia.js**, and **TypeScript**.
- Powered by a robust backend using **Laravel 12** and **PHP 8.2+**.
- Provides a comprehensive administrative panel using Spatie Permissions.
- Features real-time calculators and interactive maps.

Use the navigation bar and search functionality to explore the documentation.

2. Setup Instructions

This guide walks you through setting up the Local Development Environment for the platform.

2.1 Prerequisites

Ensure you have the following installed on your machine:

- **PHP** >= 8.2
- **Composer**
- **Node.js** (v20+ recommended) & **npm**

2.2 Installation Steps

1. **Clone the repository**

```
git clone <repo-url> nbt-price-comparison
cd nbt-price-comparison
```

2. **Install PHP dependencies**

```
composer install
```

3. **Install Node.js dependencies**

```
npm install
```

4. **Environment Configuration** Copy the example environment variables file and configure your local settings:

```
cp .env.example .env
```

Note: Ensure the default SQLite DB or your local DB credentials are correct.

5. **Generate Application Key**

```
php artisan key:generate
```

6. **Database Migration and Seeding** Create the SQLite database file if required, then migrate the database schema and seed the initial data:

```
touch database/database.sqlite
php artisan migrate --seed
```

7. **Link Storage** Needed for Spatie Media Library and uploaded assets (like avatars, logos, and post images).

```
php artisan storage:link
```

2.3 Running the Application

You can use the built-in Composer script which concurrently runs Vite, the Laravel Server, queue workers, and log streams in a single tab:

```
npm run dev
# OR using the composer wrapper:
composer dev
```

The application will be accessible at: `http://localhost:8000`

2.3.1 Server-Side Rendering (SSR)

For SSR (typically used in production or for SEO debugging), use:

```
composer dev:ssr
```

3. Architecture Overview

The system architecture blends a monolithic Laravel server with a reactive Vue SPA, bridged natively by Inertia.js.

3.1 Tech Stack

3.1.1 Backend

- **Framework**: Laravel 12.x
- **Language**: PHP 8.2+
- **Database**: SQLite (default configuration, easily switchable to MySQL/PostgreSQL)
- **Key Packages**:
- `inertiajs/inertia-laravel` for SPA routing & bridging
- `laravel/fortify` for deep authentication logic
- `spatie/laravel-permission` for roles & permissions
- `spatie/laravel-translatable` for JSON-based model translations
- `spatie/laravel-medialibrary` for handling media uploads effortlessly

3.1.2 Frontend

- **Framework**: Vue 3 (Composition API / `<script setup>`)
- **Routing**: Inertia.js (controlled by server routes)
- **Build Tool**: Vite
- **Styling**: Bootstrap 5 + custom CSS/SCSS
- **Language**: TypeScript & JavaScript
- **Key Packages**:
- `bootstrap` (UI Components)
- `@amcharts/amcharts5` (Charts and Visualizations)
- `leaflet` (Map features for ATMs/Branches)
- `@editorjs/editorjs` (Rich text content editing for News/Blog)

3.2 Directory Structure

A brief overview of the essential directories mapped to our architecture:

```
nbt-price-comparison/
├── app/
│   ├── Http/Controllers/   # Backend endpoints (API & Web)
│   ├── Models/             # Eloquent Models (User, Product, Organization, etc.)
│   └── Helpers/            # Custom helpers (e.g., LocaleHelper)
├── database/
│   ├── migrations/         # Database schema
│   └── seeders/            # Initial database population logic
├── routes/
│   ├── web.php             # Main web & localized routes
│   └── admin.php           # Admin panel routes
├── resources/
│   ├── js/
│   │   ├── Pages/          # Vue/Inertia Page Components
│   │   ├── Components/     # Reusable Vue Components
│   │   └── Layouts/        # Global Layout Wrappers (AdminLayout, AppLayout)
│   └── css/                # Global Stylesheets
```

```
├── public/                 # Compiled assets and uploads
└── docs/                   # Developer documentation files (MkDocs)
```

4. Features Overview

This page breaks down the primary sub-systems governing the NBT Price Comparison platform.

4.1 1. Localization & RTL

Full multilingual support (`en`, `ru`, `tg`) built precisely into the routing, eloquent models, and UI. Handled deeply via the custom `LocaleHelper` and Spatie Translatable. Locale routing is mandated via middleware for most client-facing routes.

4.2 2. Financial Products Catalog

A catalog logic engineered to parse, check, and compare different typed financial products (`credits`, `insurance`, `deposits`). Administrators can categorize them securely via the backend.

4.3 3. Dynamic Calculators

Integrated Loan, Credit, and Deposit calculators implemented entirely in Vue reactive states combined with API calculations when necessary. Highly scalable format.

4.4 4. Exchange Rates Manager

A dedicated currency exchange module that retrieves and computes exchange data visually, offering immediate conversions mimicking real-life exchange boards.

4.5 5. ATMs & Branches Locator

Leverages `DeviceLocation` models heavily tied to robust maps using **Leaflet.js** to pinpoint bank branches or kiosks globally (or regionally). Employs clustering for heavy data points.

4.6 6. Ratings & Application Submission (Reviews)

System for clients to analyze bank performances and submit applications / reviews natively into the platform for moderation.

4.7 7. Role-Based Admin Panel

A highly secured backend view managed completely within Inertia wrapped views. It utilizes **Spatie Permissions** to filter access, manage incoming organizations, dictate products out in the wild, and oversee applications.

5. API & Routing

5.1 Routing Architecture

All client-facing routes are wrapped inside a locale matcher middleware in `routes/web.php`.

5.1.1 Route Prefixes

- Dynamic localized routes example: `/{locale}/credits` routes to the `CreditsController` while mapping translations cleanly before controller execution.
- Valid locales are `en`, `ru`, and `tg`.
- The system prevents missing locale 404s by forcing redirect patterns to default locales or injecting them conditionally.

5.2 Client-Side State & SSR

Data hydration is completely invisible to the frontend. States are injected from Laravel Controllers directly into Vue Pages as raw reactive props via Inertia (`Inertia::render()`).

State mutations run through Inertia forms handling JSON/FormData headers implicitly.

5.3 Third-Party Scripts & API Exts

- The exchange module inherently reaches out via a Scheduled Command or standard controller ping to fetch current NBT (National Bank) metrics or customized JSON sources.
- Maps require interaction with raw coordinate JSON data dumped to the client from the backend.
- Use `npm run lint` and `npm run format` (managed by ESLint/Prettier) when adjusting these APIs.